

```

import Metal
import Foundation

// MARK: - Loader contract
//
// The manager owns the buffers and does
// the reads itself, so the loader only
// needs to (a) hand out the open file
// descriptor for paged_experts.bin and
// (b) resolve a tensor name to its
// (offset, length) from the manifest.
public protocol ExpertLocating {
    /// File descriptor for
    paged_experts.bin, opened O_RDONLY (ideally
    F_NOCACHE).
    var fileDescriptor: Int32 { get }
    /// Byte offset + length for a routed
    expert tensor, or nil if unknown.
    func location(of tensorName: String) ->
    (offset: off_t, length: Int)?
    /// Largest expert byte length in the
    manifest – used to size pool buffers.
    var maxExpertByteLength: Int { get }
}

public enum StreamingError: Error {
    case unknownTensor(String)
    case shortRead(expected: Int, got: Int)
    case ioFailed(errno: Int32)
}

```

```

// MARK: - Lease
//
// A lease is a promise that `buffer` holds
// `tensorName`'s weights and will stay
// valid until the command buffer it was
// acquired against completes on the GPU.
public struct ExpertLease: Sendable {
    public let buffer: MTLBuffer
    public let byteLength: Int
    fileprivate let slot: Int
}

// MARK: - Manager

public actor StreamingExpertManager {

    private let device: MTLDevice
    private let loader: ExpertLocating
    private let fd: Int32
    private let ioQueue =
DispatchQueue(label: "com.flex.engine.io",

qos: .userInitiated,

attributes: .concurrent)

    // Fixed pool. The whole pool doubles
    // as the LRU cache: a slot holding a hot
    // expert simply keeps getting cache
    // hits and is never evicted while hotter
    // than its neighbours. There is no
    // separate "ring buffer" vs "cache" – one
    // pool, one policy.

```

```

private let pool: [MTLBuffer]
private let slotBytes: Int

private enum SlotState: Equatable {
    case free
    case resident(String) // holds
this expert's weights
}
private var state: [SlotState]
private var inFlight: [Int] //
GPU refcount per slot; >0 means "do not
touch"
private var residentBy: [String: Int]
// expert name -> slot (the cache index)
private var lru: [Int] = [] //
slot indices, most-recently-used at the end

// Callers that miss and find every
slot busy park here until a slot frees.
private var waiters:
[CheckedContinuation<Void, Never>] = []

// Lightweight hotness telemetry (feeds
the adaptive policy discussed below).
public private(set) var hitCount = 0
public private(set) var missCount = 0
private var frequency: [String: Int] =
[:]

public init(loader: ExpertLocating,
device: MTLDevice, poolSlots: Int) {
    self.device = device
    self.loader = loader
    self.fd = loader.fileDescriptor

```

```

        self.slotBytes =
loader.maxExpertByteLength

        var buffers: [MTLBuffer] = []
        buffers.reserveCapacity(poolSlots)
        for _ in 0..

```

```

    public func acquire(_ tensorName:
String,
                        for commandBuffer:
MTLCommandBuffer) async throws ->
ExpertLease {
    frequency[tensorName, default: 0]
+= 1

    // 1. Cache hit – expert already
resident.
    if let slot =
residentBy[tensorName] {
        hitCount += 1
        touch(slot)
        return retain(slot, name:
tensorName, on: commandBuffer)
    }

    // 2. Miss – reserve a reclaimable
slot (may suspend under memory pressure).
    missCount += 1
    let slot = await reclaimSlot()

    guard let (offset, length) =
loader.location(of: tensorName) else {
        state[slot] = .free
        wakeOneWaiter()
        throw
StreamingError.unknownTensor(tensorName)
    }

    // 3. Read off-actor so a multi-
millisecond pread never blocks other
    //    acquirers. The slot is

```

```

reserved (.resident) up front so nobody
else
    //    grabs it while we read.
    state[slot] = .resident(tensorName)
    let ptr = pool[slot].contents()
    do {
        try await blockingRead(into:
ptr, offset: offset, length: length)
    } catch {
        state[slot] = .free
        wakeOneWaiter()
        throw error
    }

    residentBy[tensorName] = slot
    touch(slot)
    return retain(slot, name:
tensorName, on: commandBuffer)
}

/// Pin an expert as permanently
resident (e.g. shared experts, or the
/// globally-hot experts identified by
offline profiling). Pinned slots are
/// never chosen for eviction because
their inFlight is kept  $\geq 1$  conceptually;
/// here we simply exclude them from
the LRU eviction scan.
private var pinned: Set<Int> = []
public func pin(_ tensorName: String,
for commandBuffer: MTLCommandBuffer) async
throws {
    let lease = try await
acquire(tensorName, for: commandBuffer)

```

```

        pinned.insert(lease.slot)
    }

    public fun cacheStats() -> (hits: Int,
misses: Int, hitRate: Double) {
        let total = hitCount + missCount
        return (hitCount, missCount, total
== 0 ? 0 : Double(hitCount) /
Double(total))
    }

    // MARK: Internals

    private fun retain(_ slot: Int, name:
String, on commandBuffer: MTLCommandBuffer)
-> ExpertLease {
        inFlight[slot] += 1
        // Fence: when the GPU finishes
this command buffer, the slot is free to
reuse.
        commandBuffer.addCompletedHandler {
[weak self] _ in
            Task { await
self?.complete(slot) }
        }
        return ExpertLease(buffer:
pool[slot], byteLength: loader.location(of:
name)?.length ?? slotBytes, slot: slot)
    }

    private fun complete(_ slot: Int) {
        inFlight[slot] = max(0,
inFlight[slot] - 1)
        if inFlight[slot] == 0 {

```

```

wakeOneWaiter() }
    }

    /// Return a slot that is safe to load
    into: a free one, else the least-recently
    /// used resident slot with no GPU work
    outstanding. If none exists, suspend
    /// until a completion handler frees
    one, then retry.
    private func reclaimSlot() async -> Int
    {
        while true {
            if let free =
state.firstIndex(of: .free) { return free }
            // Evict the LRU resident slot
            that is idle and not pinned.
            if let victim =
lru.first(where: { inFlight[$0] == 0 &&
!pinned.contains($0) }) {
                if case .resident(let name)
= state[victim] { residentBy[name] = nil }
                lru.removeAll { $0 ==
victim }
                state[victim] = .free
                return victim
            }
            // Everything is in flight –
            park until a GPU completion frees a slot.
            await withCheckedContinuation {
(c: CheckedContinuation<Void, Never>) in
                waiters.append(c)
            }
        }
    }
}

```

```

        private func wakeOneWaiter() {
            guard !waiters.isEmpty else {
                return }
            waiters.removeFirst().resume()
        }

        private func touch(_ slot: Int) {
            lru.removeAll { $0 == slot }
            lru.append(slot) // most-
recently-used at the tail
        }

        /// Blocking pread, bridged onto the
        concurrent I/O queue so the actor is never
        /// stalled. We capture the raw pointer
        (Sendable) rather than the MTLBuffer.
        private func blockingRead(into ptr:
        UnsafeMutableRawPointer,
                                offset:
        off_t, length: Int) async throws {
            let fd = self.fd
            try await
            withCheckedThrowingContinuation { (cont:
            CheckedContinuation<Void, Error>) in
                ioQueue.async {
                    var total = 0
                    while total < length {
                        let n = pread(fd, ptr +
                        total, length - total, offset +
                        off_t(total))
                        if n < 0 {
                            cont.resume(throwing:
                            StreamingError.ioFailed(errno: errno));
                        }
                    }
                }
            }
        }
    }
}

```

```

return }

                if n == 0 {
cont.resume(throwing:
StreamingError.shortRead(expected: length,
got: total)); return }
                total += n
                }
                cont.resume()
            }
        }
    }
}

// MARK: - RAM-aware auto-tuning

public struct EngineConfig: Sendable {
    public let modelTier: String        //
which .gturbo package to load
    public let poolSlots: Int          //
resident expert-cache depth
    public let residentBudgetGB: Double

    /// Pick a model + cache depth from the
machine's physical RAM. Leaves a fixed
    /// headroom for macOS and the app; the
rest funds the expert pool.
    public static func
autoTune(expertBytes: Int) -> EngineConfig
{
        let ramGB =
Double(ProcessInfo.processInfo.physicalMemo
ry) / 1_073_741_824.0
        let headroomGB = 3.5
        let usableGB = max(0.5, ramGB -

```

```

headroomGB)

    let tier: String
    let residentCoreGB: Double
    switch ramGB {
        case ..<9: tier = "qwen1.5-moe-
a2.7b-4bit"; residentCoreGB = 1.2 // 8 GB
Macs
        case ..<17: tier = "moe-16b-
4bit"; residentCoreGB = 2.5 //
16 GB
        case ..<33: tier = "moe-35b-
4bit"; residentCoreGB = 4.0 //
32 GB
        default: tier = "moe-80b-
4bit"; residentCoreGB = 7.0 //
64 GB+
    }

    let cacheBudgetGB = max(0.25,
usableGB - residentCoreGB)
    let slots = max(4,
Int((cacheBudgetGB * 1_073_741_824.0) /
Double(expertBytes)))
    return EngineConfig(modelTier:
tier, poolSlots: slots, residentBudgetGB:
residentCoreGB)
}
}

```